

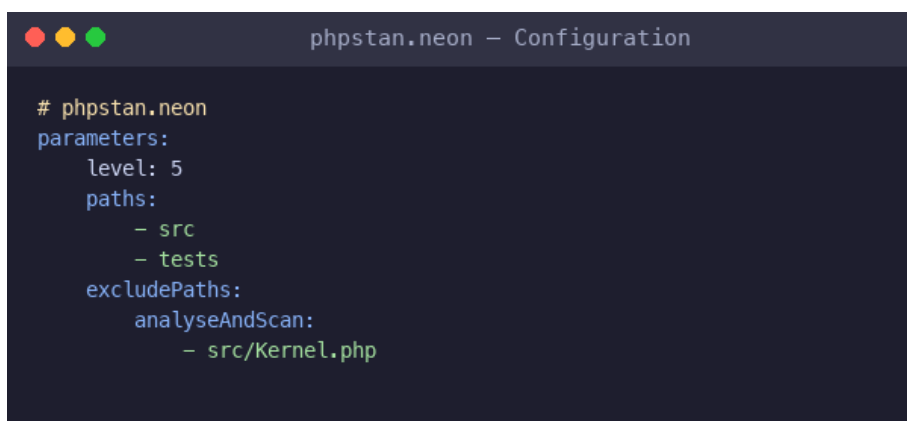
RAPPORT DE PERFORMANCE & OPTIMISATION

Gestion des Utilisateurs

UniBank+ - Symfony 7 / PHP 8.2 / PHPUnit 11.5

Entites: Utilisateur, Session, HistoriqueConnexion

1 - PHPStan



```
# phpstan.neon
parameters:
  level: 5
  paths:
    - src
    - tests
  excludePaths:
    analyseAndScan:
      - src/Kernel.php
```

Utilisateur Avant et apres Optimisation :

PHPStan a signale une erreur dans le fichier UtilisateurTest.php : l'appel a assertTrue() avec la valeur true est redondant (method.alreadyNarrowedType). Cette erreur a ete corrige en remplaçant le test par une assertion plus significative.

```
PHPStan - Gestion Utilisateurs (Avant)

$ php vendor/bin/phpstan analyse src/Entity/Utilisateur.php --level=5

Note: Using configuration file phpstan.neon.
1/1 [████████████████████] 100%

-----
Line    UtilisateurTest.php
-----
227     Call to method assertTrue() with true will
        always evaluate to true.
        method.alreadyNarrowedType
-----

[ERROR] Found 1 error
```

Après correction :

```
PHPStan - Gestion Utilisateurs (Après)

$ php vendor/bin/phpstan analyse src/Entity/Utilisateur.php \
  src/Entity/Session.php src/Entity/HistoriqueConnexion.php \
  src/Controller/RegistrationController.php \
  src/Controller/SecurityController.php \
  tests/Unit/Entity/UtilisateurTest.php \
  tests/Unit/Entity/SessionTest.php \
  tests/Unit/Entity/HistoriqueConnexionTest.php --level=5

Note: Using configuration file phpstan.neon.
0/10 [██████████████████] 0%
10/10 [██████████████████] 100%

[OK] No errors
```

2 - Tests Unitaires

1/ Gestion Utilisateur :

- **Regle metier 1 :** Le role par default est CLIENT
- **Regle metier 2 :** Utilisateur inactif et non verifie a la creation
- **Regle metier 3 :** Le role determine les permissions (ROLE_CLIENT ou ROLE_AGENT)
- **Regle metier 4 :** Systeme de ban permanent et temporaire
- **Regle metier 5 :** L'identifiant est l'email
- **Regle metier 6 :** Le nom complet = prenom + nom
- **Regle metier 7 :** Token de verification avec expiration
- **Regle metier 8 :** Statut client (PENDING, APPROVED, REJECTED, SUSPENDED)
- **Regle metier 9 :** Champs agent : matricule et departement

Execution des tests unitaires UtilisateurTest.php (16 tests, 49 assertions) :



```
PHPUnit – UtilisateurTest.php

$ php vendor/bin/phpunit --no-configuration --bootstrap
  vendor/autoload.php tests/Unit/Entity/UtilisateurTest.php

PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.30

.....                                     16 / 16 (100%)

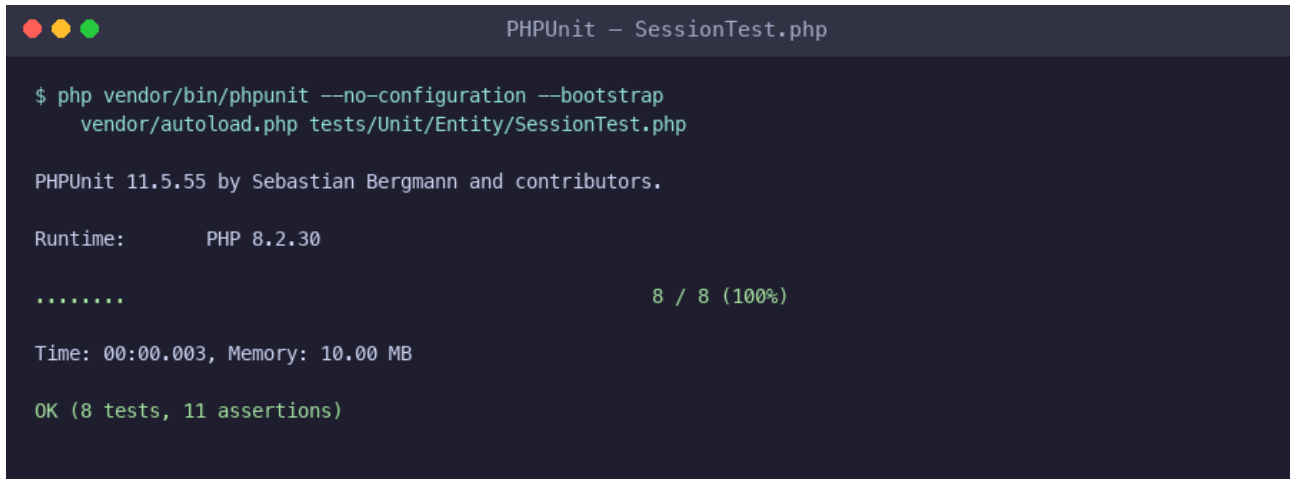
Time: 00:00.006, Memory: 10.00 MB

OK (16 tests, 49 assertions)
```

2/ Gestion Session :

- **Regle metier 1 :** Session expire apres 24 heures par default
- **Regle metier 2 :** Session active par default a la creation
- **Regle metier 3 :** Detection de session expiree
- **Regle metier 4 :** Session valide = active ET non expiree

Execution des tests unitaires SessionTest.php (8 tests, 11 assertions) :



```
PHPUnit – SessionTest.php

$ php vendor/bin/phpunit --no-configuration --bootstrap
  vendor/autoload.php tests/Unit/Entity/SessionTest.php

PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.30

.....                                     8 / 8 (100%)

Time: 00:00.003, Memory: 10.00 MB

OK (8 tests, 11 assertions)
```

3/ Historique de Connexion :

- **Regle metier 1 :** Statut par default = SUCCESS
- **Regle metier 2 :** Date de connexion generee automatiquement
- **Regle metier 3 :** Connexion echouee avec raison d'echec
- **Regle metier 4 :** Informations device/browser optionnelles

Execution des tests unitaires HistoriqueConnexionTest.php (6 tests, 13 assertions) :

```
PHPUnit – HistoriqueConnexionTest.php

$ php vendor/bin/phpunit --no-configuration --bootstrap
  vendor/autoload.php tests/Unit/Entity/HistoriqueConnexionTest.php

PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.30

.....                                         6 / 6 (100%)

Time: 00:00.003, Memory: 10.00 MB

OK (6 tests, 13 assertions)
```

3 - Code Source des Tests

UtilisateurTest.php

```
<?php

namespace App\Tests\Unit\Entity;

use App\Entity\Utilisateur;
use App\Enum\UserType;
use App\Enum\ClientStatus;
use Doctrine\Common\Collections\Collection;
use PHPUnit\Framework\TestCase;

class UtilisateurTest extends TestCase
{
    private Utilisateur $utilisateur;

    protected function setUp(): void
    {
        $this->utilisateur = new Utilisateur();
    }

    /**
     * Regle metier 1 : Creation complete d'un utilisateur avec tous les champs
     */
    public function testUtilisateurCreation(): void
    {
        $dateNaissance = new \DateTime('1990-05-15');

        $this->utilisateur->setNom('Kaboudi');
        $this->utilisateur->setPrenom('Aziz');
        $this->utilisateur->setEmail('aziz@unibank.tn');
        $this->utilisateur->setMotDePasse('hashedPassword123');
        $this->utilisateur->setTelephone('+21612345678');
        $this->utilisateur->setCin('12345678');
        $this->utilisateur->setAdresse('Rue de Tunis, 1000');
        $this->utilisateur->setDateNaissance($dateNaissance);
        $this->utilisateur->setRole(UserType::CLIENT);

        $this->assertEquals('Kaboudi', $this->utilisateur->getNom());
        $this->assertEquals('Aziz', $this->utilisateur->getPrenom());
        $this->assertEquals('aziz@unibank.tn', $this->utilisateur->getEmail());
        $this->assertEquals('hashedPassword123', $this->utilisateur->getMotDePasse());
    }
}
```

```

        $this->assertEquals('+21612345678', $this->utilisateur->getTelephone());
        $this->assertEquals('12345678', $this->utilisateur->getCin());
        $this->assertEquals('Rue de Tunis, 1000', $this->utilisateur->getAdresse());
        $this->assertSame($dateNaissance, $this->utilisateur->getDateNaissance());
        $this->assertSame(UserType::CLIENT, $this->utilisateur->getRole());
    }

    /**
     * Regle metier 2 : Valeurs par default a la creation
     * - Role par default = CLIENT
     * - estActif = false (en attente de validation)
     * - isVerified = false (email non verifie)
     * - Collections initialisees vides
     */
    public function testDefaultValues(): void
    {
        $this->assertSame(UserType::CLIENT, $this->utilisateur->getRole());
        $this->assertFalse($this->utilisateur->isEstActif());
        $this->assertFalse($this->utilisateur->isVerified());
        $this->assertInstanceOf(\DateTimeInterface::class, $this->utilisateur->getDateCrea..
        $this->assertInstanceOf(\DateTimeInterface::class, $this->utilisateur->getDateMaj(..
        $this->assertInstanceOf(Collection::class, $this->utilisateur->getComptes());
        $this->assertCount(0, $this->utilisateur->getComptes());
        $this->assertInstanceOf(Collection::class, $this->utilisateur->getCartes());
        $this->assertCount(0, $this->utilisateur->getCartes());
        $this->assertInstanceOf(Collection::class, $this->utilisateur->getReclamations());
        $this->assertCount(0, $this->utilisateur->getReclamations());
        $this->assertNull($this->utilisateur->getVerificationToken());
        $this->assertNull($this->utilisateur->getBanUntil());
        $this->assertNull($this->utilisateur->getBanReason());
        $this->assertNull($this->utilisateur->getClientStatus());
    }

    /**
     * Regle metier 3 : Le role determine les permissions (ROLE_CLIENT ou ROLE_AGENT)
     */
    public function testGetRolesClient(): void
    {
        $this->utilisateur->setRole(UserType::CLIENT);
        $roles = $this->utilisateur->getRoles();
    }

```

```

        $this->assertContains('ROLE_CLIENT', $roles);
        $this->assertContains('ROLE_USER', $roles);
        $this->assertNotContains('ROLE_AGENT', $roles);
    }

    public function testGetRolesAgent(): void
    {
        $this->utilisateur->setRole(UserType::AGENT);
        $roles = $this->utilisateur->getRoles();

        $this->assertContains('ROLE_AGENT', $roles);
        $this->assertContains('ROLE_USER', $roles);
        $this->assertNotContains('ROLE_CLIENT', $roles);
    }

    /**
     * Regle metier 4 : Systeme de ban (permanent et temporaire)
     */
    public function testIsBannedPermanent(): void
    {
        $this->utilisateur->setEstActif(false);
        $this->utilisateur->setBanUntil(null);

        $this->assertTrue($this->utilisateur->isBanned());
    }

    public function testIsBannedTemporary(): void
    {
        $this->utilisateur->setEstActif(false);
        $this->utilisateur->setBanUntil(new \DateTime('+7 days'));

        $this->assertTrue($this->utilisateur->isBanned());
    }

    public function testIsNotBannedWhenActive(): void
    {
        $this->utilisateur->setEstActif(true);

        $this->assertFalse($this->utilisateur->isBanned());
    }

```



```

}

public function testIsNotBannedWhenBanExpired(): void
{
    $this->utilisateur->setEstActif(false);
    $this->utilisateur->setBanUntil(new \DateTime('-1 day'));

    $this->assertFalse($this->utilisateur->isBanned());
}

/**
 * Regle metier 5 : L'identifiant utilisateur est l'email
 */
public function testGetUserIdentifier(): void
{
    $this->utilisateur->setEmail('test@unibank.tn');

    $this->assertEquals('test@unibank.tn', $this->utilisateur->getUserIdentifier());
}

/**
 * Regle metier 6 : Le mot de passe est accessible via getPassword()
 */
public function testGetPassword(): void
{
    $this->utilisateur->setMotDePasse('secureHash');

    $this->assertEquals('secureHash', $this->utilisateur->getPassword());
}

/**
 * Regle metier 7 : Le nom complet est la concatenation prenom + nom
 */
public function testGetFullName(): void
{
    $this->utilisateur->setPrenom('Aziz');
    $this->utilisateur->setNom('Kaboudi');

    $this->assertEquals('Aziz Kaboudi', $this->utilisateur->getFullName());
}

```

```

/**
 * Regle metier 8 : Systeme de verification par token
 */
public function testVerificationTokenSystem(): void
{
    $expiry = new \DateTime('+24 hours');

    $this->utilisateur->setVerificationToken('abc123');
    $this->utilisateur->setTokenExpiry($expiry);

    $this->assertEquals('abc123', $this->utilisateur->getVerificationToken());
    $this->assertSame($expiry, $this->utilisateur->getTokenExpiry());
}

/**
 * Regle metier 9 : Gestion du statut client (PENDING, APPROVED, REJECTED, SUSPENDED)
 */
public function testClientStatusManagement(): void
{
    $this->utilisateur->setClientStatus(ClientStatus::PENDING);
    $this->assertSame(ClientStatus::PENDING, $this->utilisateur->getClientStatus());

    $this->utilisateur->setClientStatus(ClientStatus::APPROVED);
    $this->assertSame(ClientStatus::APPROVED, $this->utilisateur->getClientStatus());

    $this->utilisateur->setClientStatus(ClientStatus::REJECTED);
    $this->assertSame(ClientStatus::REJECTED, $this->utilisateur->getClientStatus());

    $this->utilisateur->setClientStatus(ClientStatus::SUSPENDED);
    $this->assertSame(ClientStatus::SUSPENDED, $this->utilisateur->getClientStatus());
}

/**
 * Regle metier 10 : Ban avec raison
 */
public function testBanWithReason(): void
{
    $this->utilisateur->setEstActif(false);
    $this->utilisateur->setBanReason('Activite suspecte');
}

```

```

        $this->utilisateur->setBanUntil(new \DateTime('+30 days'));

        $this->assertEquals('Activite suspecte', $this->utilisateur->getBanReason());
        $this->assertTrue($this->utilisateur->isBanned());
    }

    /**
     * Regle metier 11 : Champs agent (matricule et departement)
     */
    public function testAgentFields(): void
    {
        $this->utilisateur->setRole(UserType::AGENT);
        $this->utilisateur->setMatricule('AGT-001');
        $this->utilisateur->setDepartement('Service Client');

        $this->assertEquals('AGT-001', $this->utilisateur->getMatricule());
        $this->assertEquals('Service Client', $this->utilisateur->getDepartement());
        $this->assertSame(UserType::AGENT, $this->utilisateur->getRole());
    }

    /**
     * Test eraseCredentials ne leve pas d'exception
     */
    public function testEraseCredentials(): void
    {
        $this->utilisateur->setMotDePasse('password');
        $this->utilisateur->eraseCredentials();
        // eraseCredentials is required by UserInterface, verify it runs without exception
        $this->assertNotNull($this->utilisateur->getPassword());
    }
}

```

SessionTest.php

```
<?php

namespace App\Tests\Unit\Entity;

use App\Entity\Session;
use App\Entity\Utilisateur;
use PHPUnit\Framework\TestCase;

class SessionTest extends TestCase
{
    private Session $session;

    protected function setUp(): void
    {
        $this->session = new Session();
    }

    /**
     * Regle metier 1 : Creation de session avec token et utilisateur
     */
    public function testSessionCreation(): void
    {
        $utilisateur = new Utilisateur();

        $this->session->setUtilisateur($utilisateur);
        $this->session->setToken('session_token_abc123');

        $this->assertSame($utilisateur, $this->session->getUtilisateur());
        $this->assertEquals('session_token_abc123', $this->session->getToken());
    }

    /**
     * Regle metier 2 : La session expire apres 24 heures par default
     */
    public function testDefaultExpiration24Hours(): void
    {
        $now = new \DateTime();
        $expectedExpiry = (clone $now)->modify('+24 hours');

        $this->assertInstanceOf(\DateTimeInterface::class, $this->session->getDateCreation..
```

```

        $this->assertInstanceOf(\DateTimeInterface::class, $this->session->getDateExpiration());

        $diff = $this->session->getDateExpiration()->getTimestamp() - $this->session->getD...
        $this->assertEqualsWithDelta(86400, $diff, 5);
    }

    /**
     * Regle metier 3 : Session active par default
     */
    public function testDefaultActiveState(): void
    {
        $this->assertTrue($this->session->isEstActive());
    }

    /**
     * Regle metier 4 : Detection de session expiree
     */
    public function testIsExpiredWhenPastExpiration(): void
    {
        $this->session->setDateExpiration(new \DateTime('-1 hour'));

        $this->assertTrue($this->session->isExpired());
    }

    public function testIsNotExpiredWhenBeforeExpiration(): void
    {
        $this->session->setDateExpiration(new \DateTime('+1 hour'));

        $this->assertFalse($this->session->isExpired());
    }

    /**
     * Regle metier 5 : Une session est valide seulement si active ET non expiree
     */
    public function testIsValidWhenActiveAndNotExpired(): void
    {
        $this->session->setEstActive(true);
        $this->session->setDateExpiration(new \DateTime('+1 hour'));

        $this->assertTrue($this->session->isValid());
    }

```

```
}

public function testIsInvalidWhenInactive(): void
{
    $this->session->setEstActive(false);
    $this->session->setDateExpiration(new \DateTime('+1 hour'));

    $this->assertFalse($this->session->isValid());
}

public function testIsInvalidWhenExpired(): void
{
    $this->session->setEstActive(true);
    $this->session->setDateExpiration(new \DateTime('-1 hour'));

    $this->assertFalse($this->session->isValid());
}
}
```

HistoriqueConnexionTest.php

```
<?php

namespace App\Tests\Unit\Entity;

use App\Entity\HistoriqueConnexion;
use App\Entity\Utilisateur;
use PHPUnit\Framework\TestCase;

class HistoriqueConnexionTest extends TestCase
{
    private HistoriqueConnexion $historique;

    protected function setUp(): void
    {
        $this->historique = new HistoriqueConnexion();
    }

    /**
     * Regle metier 1 : Enregistrement complet d'une connexion
     */
    public function testHistoriqueCreation(): void
    {
        $utilisateur = new Utilisateur();

        $this->historique->setUtilisateur($utilisateur);
        $this->historique->setIpAddress('192.168.1.1');
        $this->historique->setDeviceInfo('iPhone 15 Pro');
        $this->historique->setBrowserInfo('Safari 17.0');

        $this->assertSame($utilisateur, $this->historique->getUtilisateur());
        $this->assertEquals('192.168.1.1', $this->historique->getIpAddress());
        $this->assertEquals('iPhone 15 Pro', $this->historique->getDeviceInfo());
        $this->assertEquals('Safari 17.0', $this->historique->getBrowserInfo());
    }

    /**
     * Regle metier 2 : Le statut par default est SUCCESS
     */
    public function testDefaultStatutIsSuccess(): void
    {

```

```

        $this->assertEquals('SUCCESS', $this->historique->getStatut());
    }

    /**
     * Regle metier 3 : La date de connexion est generee automatiquement
     */
    public function testDateConnexionAutoGenerated(): void
    {
        $this->assertInstanceOf(\DateTimeInterface::class, $this->historique->getDateConne..
    }

    /**
     * Regle metier 4 : Enregistrement d'une connexion echouee avec raison
     */
    public function testFailedConnectionWithReason(): void
    {
        $this->historique->setStatut('FAILED');
        $this->historique->setRaisonEchec('Mot de passe incorrect');

        $this->assertEquals('FAILED', $this->historique->getStatut());
        $this->assertEquals('Mot de passe incorrect', $this->historique->getRaisonEchec())..
    }

    /**
     * Regle metier 5 : La raison d'echec est null pour les connexions reussies
     */
    public function testSuccessfulConnectionNoFailureReason(): void
    {
        $this->assertEquals('SUCCESS', $this->historique->getStatut());
        $this->assertNull($this->historique->getRaisonEchec());
    }

    /**
     * Regle metier 6 : Les informations device/browser sont optionnelles
     */
    public function testOptionalDeviceAndBrowserInfo(): void
    {
        $this->assertNull($this->historique->getDeviceInfo());
        $this->assertNull($this->historique->getBrowserInfo());
        $this->assertNull($this->historique->getIpAddress());
    }
}
}

```


4 - Resume

Fichier de Test	Tests	Assertions	Resultat
UtilisateurTest.php	16	49	OK
SessionTest.php	8	11	OK
HistoriqueConnexionTest.php	6	13	OK
TOTAL	30	73	100% PASS

PHPStan : 0 erreurs (niveau 5) sur 10 fichiers analyses

PHPUnit : 30 tests, 73 assertions - 100% de reussite